Game Programming 01

Introduction **&&** Unity Basics

Overview

00 - **Assignments and Project**

> One Assignment per week (total: 5)

> Integration into Game Production Project

> Goal: write your own code!

> Submission through Github Classroom

```
using UnityEngine;

0 references
public class Player : MonoBehaviour
{

    // Start is called before the first frame update
    0 references
    void Start()
    {

    }

    // Update is called once per frame
    0 references
    void Update()
    {

    }
}
```

Overview Topics

01 - **Introduction && Unity Basics**

> Overview

> Basics Unity and Unity Programming

02 - **Delegates, Events, etc.**

> Dependency Best Practices

> Event Handling approaches

> Trigger/Collision Handling
Rigidbody, 2D vs 3D
Tags && Layers

Overview Topics

03 - **Visuals**

> Creation of Materials

> Base Map, Normals, Emission, ...

> URP Shader Graph

> Lighting, Particle Systems, Post Processing

04 - **Audio System**

> Audio Handling in Unity

> Clips, Audio Sources, Audio Mixer, ...

> Implementation of a Sound System

> Scriptable Objects

Overview Topics

05 - **Topic of Choice**

> dependent on what you need or want to learn

> e.g. Animations, Cinemachine, Input Handling, Dialogue System, UI, Terrain Tool, …

>> **What will come out of this?**

> Virtual Pet Rock Game!

> Working on one game during class

> Will be built upon with every Lesson

Unity && C# Basics

New project

Editor version *

6000.3.9f1 Silicon LTS

Search

All

Universal 2D

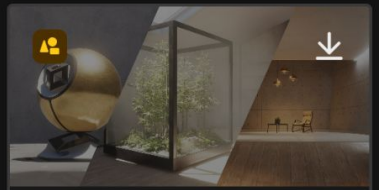
Universal 3D

High Definition 3D

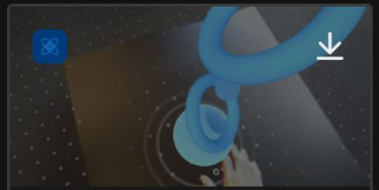
Get Started With Unity

Essentials Pathway

Universal 3D sample



High Definition 3D sample



Mixed Reality (MR)

MR Multiplayer Tabletop Template

Universal 3D

The URP (Universal Render Pipeline) blank template includes the settings and assets you need to start creating with URP. Equipped with artist-...

View details

Unity organization

unity_CDEBCCEC0DE74D0618C9

Project name *

myPetRock

Location *

/Users/samzuehlke/work/teaching/games

Source control provider

GitHub

Personal Access Token *

Connected as splendiferousnoctifer

Disconnect

Repository name *

myPetRock

Slug preview: mypetrock

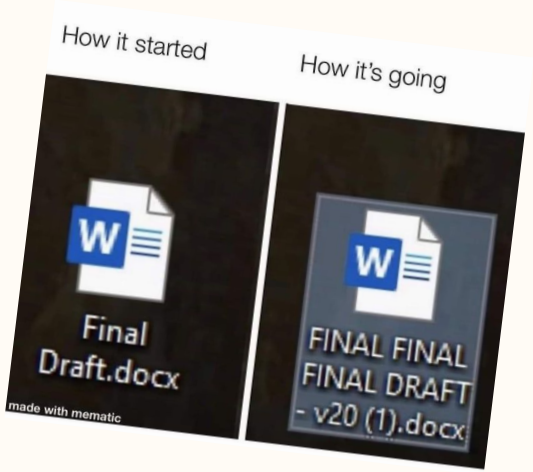
Additional configuration

Create project

Unity Game Setup

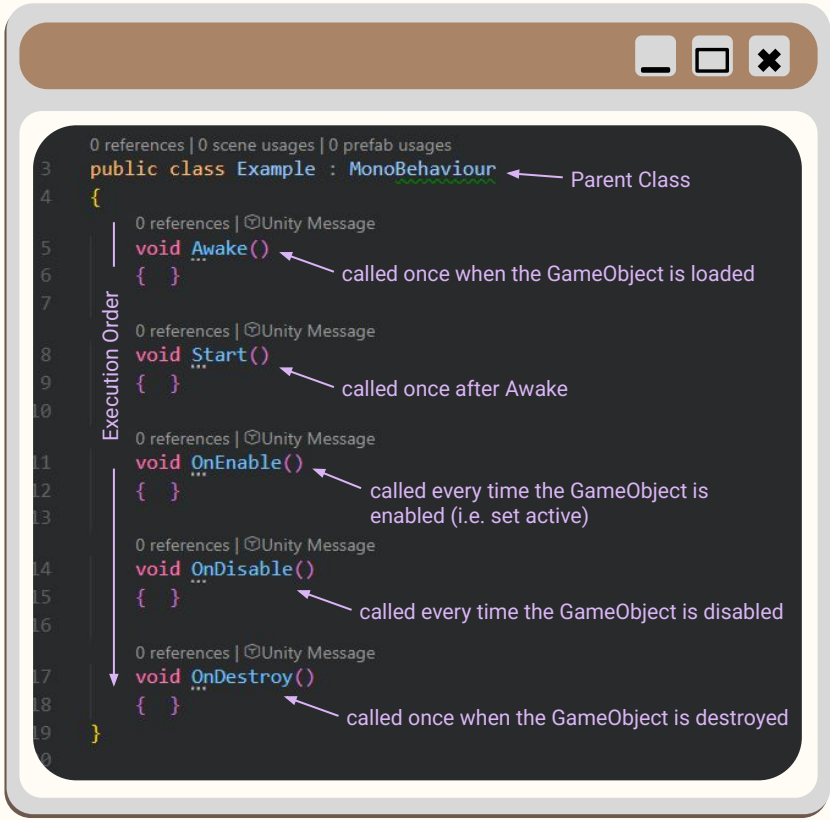
> Use the correct Unity version for the course
> Choose Universal 3D (URP) (unless your project has a specific reason not to)

> Please Name your project clearly
> Store it somewhere you can find again
> Connect GitHub / version control from the start
> Avoid spaces, umlauts, and special characters in project/folder names



Classes in C#

- Most common: using a *MonoBehaviour* Class
- *MonoBehaviour* allows us to attach the script as a component to a GameObject
- use *Start()* for tasks that depend on other GameObjects being fully initialized
- having a lot of logic in Awake/Start adds to Unity's activation time



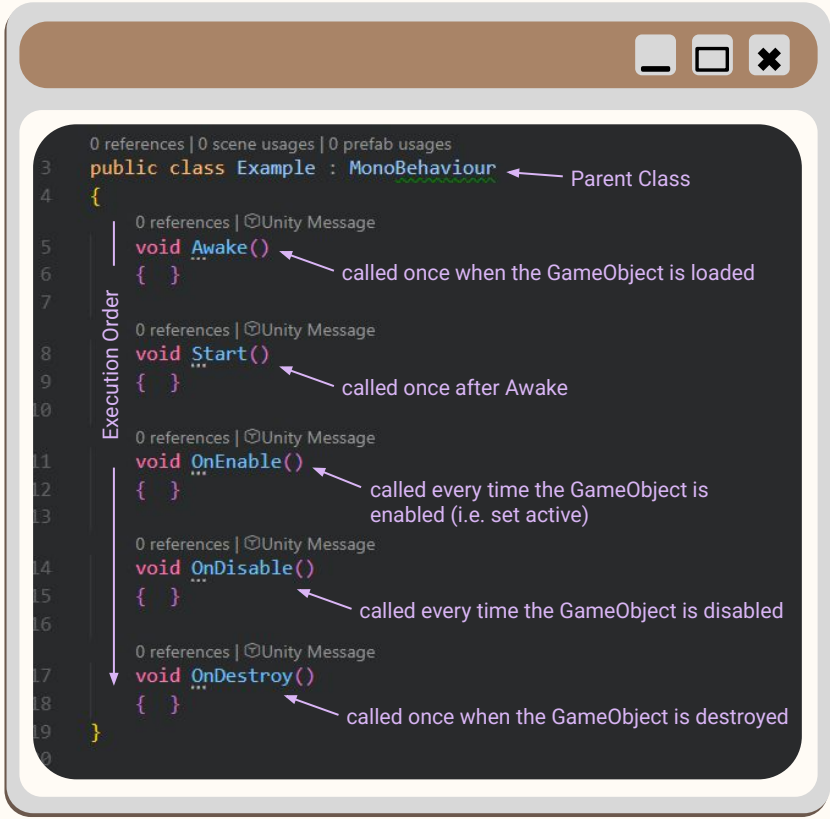
MonoBehaviour VS. Plain Class

> MonoBehaviour

- can be attached to GameObjects
- has Unity lifecycle methods
- can use Start(), Update(), OnEnable(), etc.
- visible in the Inspector

> Plain C# class

- cannot be attached to a GameObject
- useful for data, logic, systems, helper classes
- usually created with new



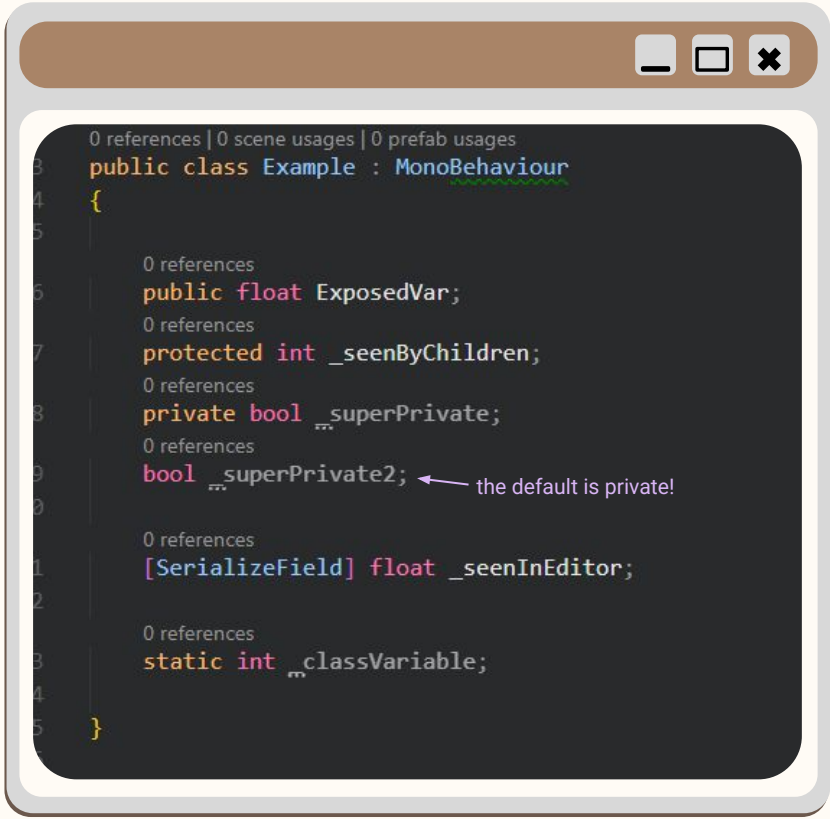
Variables and Access Modifiers

- > public: accessible from outside of script
- > private: accessible only from inside
- > (a secret third thing 👀; see later slide)

- > static: *Class Variable* instead of *Object Variable*; changing this changes it for all Objects of this Class; can be accessed only through Class Name

- > SerializeField: exposing to the Editor without making the variable public

- > Keep your variables/methods as private as possible!



Accessing Variables

> Instead of making your variable public, there are other possibilities:

> ✨ Getters and Setters ✨

> what you choose depends purely on what you need


```
0 references | 0 scene usages | 0 prefab usages
public class Example : MonoBehaviour
{

    4 references
    int _protectedWithMyLife;

    0 references
    public int ForOthersToSee
    {
        get { return _protectedWithMyLife; }
        private set { _protectedWithMyLife = value; }
    }

    0 references
    public int SimpleGetter => _protectedWithMyLife;
    0 references
    public int SimpleGetterModified => _protectedWithMyLife * 2;

    0 references
    public bool AnotherPossibility { get; private set; }
```

Inheritance

- to inherit the properties and methods of the parent class
- improves code by a lot!
- enables code abstraction, reusability/scalability and reduces code duplication
- protected: only this class and children can see/modify it

- *MonoBehaviour* is also inheritance! we get access to variables and methods (like *Start* and *Update*)

```
4   public abstract class DialogueBox : MonoBehaviour
5   {                                   <- you can't make objects of abstract classes!
        0 references
6       [SerializeField] protected TextMeshProUGUI _dialogueText;
        0 references
7       public virtual void StartDialogue() {}   <- virtual methods can be overridden by children
        2 references
8       public abstract void UpdateUI();
9   }                                   <- no implementation, needs to be implemented by children
10
    0 references
11  public class DialogueBoxSimple : DialogueBox
12  {
        0 references
13      [SerializeField] TextMeshProUGUI _characterName;
        1 reference
14      public override void UpdateUI() { /* implemented */ }
15  }
16
    0 references
17  public class DialogueBoxWithChoices : DialogueBox
18  {
        0 references
19      [SerializeField] Transform _choicesParent;
        1 reference
20      public override void UpdateUI() { /* implemented */ }
        0 references
21      void DisplayChoices() {}
22  }
```

>> Polymorphism

> Goes hand-in-hand with Inheritance

> Allows things like:
- methods behaving differently based on which object they're called from
- using the parent class for detection, then accessing child-specific logic
- etc.


```
List<DialogueBox> dialogueBoxes = new();
dialogueBoxes.Add(new DialogueBoxSimple());
dialogueBoxes.Add(new DialogueBoxWithChoices());
```

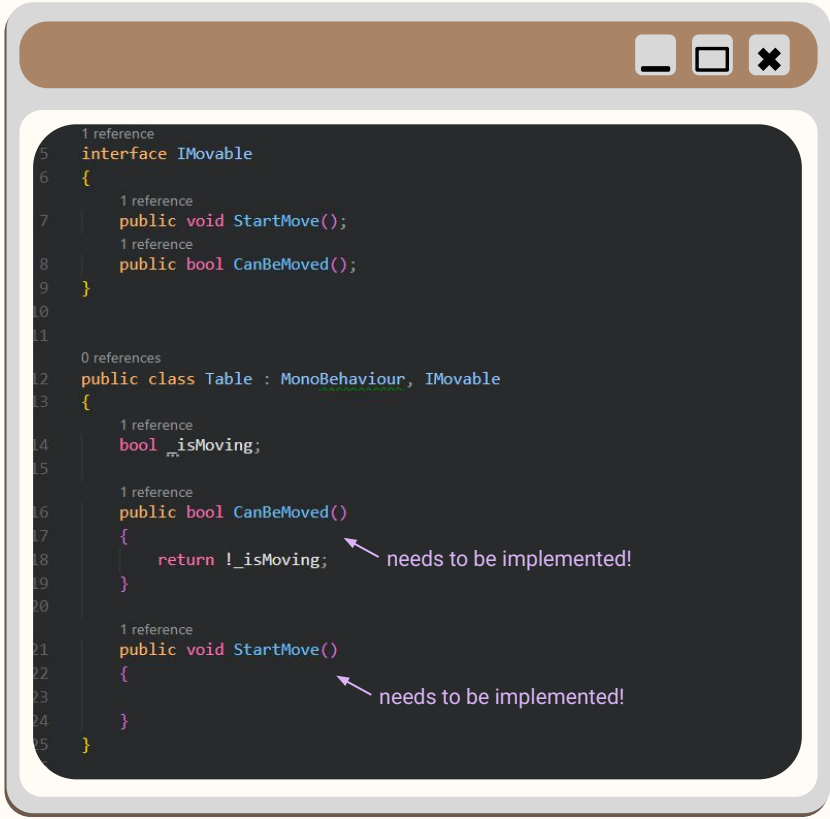
```
0 references
public class DialogueBoxSimple : DialogueBox
{
    1 reference
    [SerializeField] TextMeshProUGUI _characterName;
    1 reference
    public override void UpdateUI()
    {
        _characterName.text = "Example Name";
        _dialogueText.text = "This is something that was said (probably)";
    }
}

0 references
public class DialogueBoxWithChoices : DialogueBox
{
    0 references
    [SerializeField] Transform _choicesParent;
    1 reference
    public override void UpdateUI()
    {
        _dialogueText.text = "Completely different dialogue";
        DisplayChoices();
    }
    1 reference
    void DisplayChoices() {}
}
```

Interfaces

> Basically a completely *abstract* class that only contains abstract methods and properties

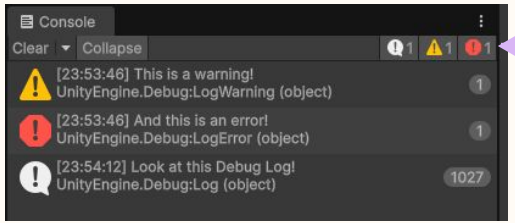
> We can only inherit from one Class, but we can implement multiple Interfaces!

> usually written like “IMovable” to distinguish it

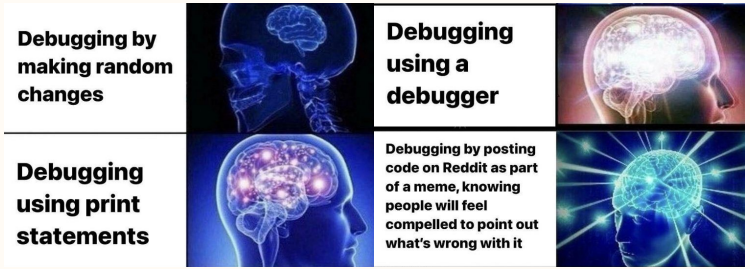


Debugging

> Debug.Log()
> Debug.LogWarning()
> Debug.LogError()
> You can filter them in the Console:




```
string exVar = "Test Test";
Debug.Log($"You can also format this and add text like for example {exVar}!");
```



Good Practices

> If something doesn't work, put a Debug.Log() :)

> If it still doesn't work, put even more :))

> At some point you'll find the issue :')

> Also good for: making your scripts usable for e.g. Game Designers

Common Beginner Errors

> **NullReferenceException**
> You tried to use something that was not assigned/found.

> **Object does not move**
> Check Transform, Rigidbody, constraints, script enabled, object active.

> **Code does not run**
> Check whether the script is attached to a GameObject and if it’s active.

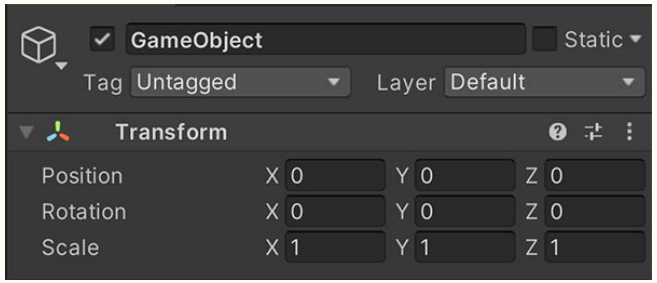
> **Changes disappear after pressing Play**
> You changed values during Play Mode.

> **Method not called**
> Check spelling: Start, Update, OnEnable, not start, update, etc.

GameObjects and their Transforms

GameObjects:

> Everything in your scene is a GameObject
> They’re simple containers you can add Components to to give them a purpose
> Can be cameras, characters, lights, …



Transform:

> Every GameObject has a *Transform* (or *RectTransform* if it’s a UI-Object)
> It can only have one
> Dictates the Object’s location, rotation, and scale

What are Components?

> “Components are the functional pieces of every GameObject.” - Unity Manual

> GameObjects consist of components
  - Transform is also just a component!
  - can be self-created Scripts, UI-things like Text, a Mesh Renderer for 3D objects, …

> Self-created scripts must inherit from *MonoBehaviour* so they can be attached to GameObjects

> You can change how the inspector display’s your component’s properties with Editor Scripting (that’s an advanced topic, but you can look it up if you’re interested :D)

>> Local vs. World Space Coordinate System

> **World Space**

Position/rotation relative to the whole scene

`transform.position`

> **Local Space**

Position/rotation relative to the parent object

`transform.localPosition`

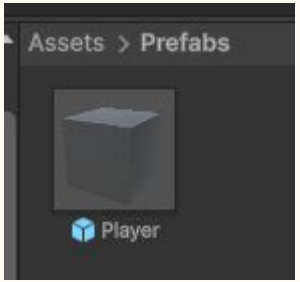
Thus, if an object has no parent, local and world position are effectively the same

Prefabs

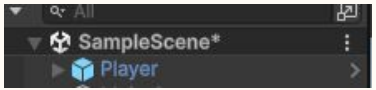
> A Prefab is a reusable GameObject template. Useful for enemies, bullets, collectibles, food, toys, UI elements, etc.
> You can place Prefabs manually in the scene
> You can also create them while the game is running

Prefab = template in the Project folder
Instance = copy placed in the scene

```
1 reference | Unity Serialized Field
[SerializeField] PetRock _petRockPrefab;

0 references
void SpawnPetRock()
{                                Prefab          Position          Rotation          Parent
    PetRock petRock = Instantiate(_petRockPrefab, transform.position, Quaternion.identity, transform);
    petRock.SomePublicMethod();
}
```



Prefabs are Assets and can exist in your Project Folder



in the Hierarchy

The Main Thread and Coroutines

Unity runs most gameplay code on the main thread, **`Update()`** is called once per frame. Long loops or blocking code can freeze the game. Do not use **`while(true)`** or long waiting loops in **`Update()`**

Coroutines allow code to pause and continue later which is useful for:

> waiting

> timed animations

> delays

> repeating actions over time

```
0 references
IEnumerator DisableSelf(float _waitTime)
{
    yield return new WaitForSeconds(_waitTime);
    gameObject.SetActive(false);
}
```

The Main Thread and Coroutines

> Coroutines are not true multithreading !

> They still run on Unity’s main thread

> They just split work across frames/time

Let's say our Pet Rock becomes hungry every 2 minutes. Instead of Pausing the entire Game for that duration, we ‘delegate’ the waiting to the coroutine. After eating, it waits before becoming hungry again.

Getting Player Input

Unity has an old and a new Input System, but we use the new Input System, because input should describe player intent, not keyboard keys directly.
(and Unity 6 has it pre-installed)

> Bad: *“Space key pressed”*

> Better: *“Jump action performed”*

```
private void Update()
{
    if (interactAction.action.WasPerformedThisFrame())
    {
        Debug.Log("Interact!");
    }
}
```

The Camera

The Camera defines *what the player sees*, every active scene needs at least one active camera.
The camera has: Position, Rotation, Field of View, Projection type, Culling Mask

Projection Types:
Perspective: realistic depth
Orthographic: flat/isometric/2D-style view

```
Camera.main.transform.LookAt(target);
```

Scene Loading

A Unity project can contain multiple scenes like the Main Menu, the Game Scene, Credits, and Test or Tutorial Scene

Please remember: Scenes must be added to Build Settings before they can be loaded by name/index

```
public void LoadGameScene()
{
    SceneManager.LoadScene("GameScene");
}
```

This runs on the main thread!
If you're loading a very big scene (which can take up to minutes), do it **asynchronously**!

Unity’s Asset Store

The Asset Store/Fab can provide:

> 3D models
> textures/materials
> animations
> audio
> scripts/tools
> particle effects

> and more!

Prefer small, simple assets for assignments

Assignment #1

Infos

> Reminder: All Assignments will be implemented in your *Game Production* Project

> Because of this, there will sometimes not be super detailed instructions

Assignment

> Look for an Asset online and add it to your project (import it and add it somewhere in the game)

> Give this Asset some Logic

> You can use the Concepts we've talked about in this lesson

[Notes for me]; **possible topics**

- create a class;
  - MonoBehaviour vs plain class
  - Inheritance basics, Interfaces?
    - private vs public vs protected
    - static
    - SerializeField (+ stuff like TextArea, Color Grabber etc.?)
    - also: Lifecycle (Awake, Start, ….)
    - explain components
- Debugging (Debug Log, Warning, Error) and how to use efficiently
- Transforms and Object Parenting (world vs local coordinate system)
- Get User Input (new InputSystem Basics?)
- Prefabs, Instantiate and Destroy
- Coroutines, Script Execution Basics (execution per frame, main thread etc.)
- Variables vs. GetComponent Function (etc.)
- Asset Store
- Camera
- Scene Loading?

[Notes for me]; **Project Idea #3**
> “Virtual Pet” (like Nintendogs)

- Basics: inheritance with diff types of food/toys, private/public/SerializeField to access toy vars, Transforms and Object Parenting (moving object with mouse), User input to maybe whistle for the animal to come to camera, prefabs + variants for different food/toys, static camera
- Events: events for taking objects and doing interactions and UI, trigger/collision when food/toy touches animal or floor, all of the toys have a Rigidbody, Trigger vs Raycast?, also Input Manager and Events
- Visuals: we can do a lot in a room, e.g. placing a bowl of water, a fireplace, …
- Audio: BGM, different sounds for different objects

Sources for Project:

- Pet Rock: https://sketchfab.com/3d-models/pet-rock-1da2916d9ee74c239eb681d344017f5a
- Room: https://sketchfab.com/3d-models/another-single-isometric-room-1cf0ee2c858746249f8ad945706e2986
- Pet Food: https://www.fab.com/listings/dc14f987-f31c-42ae-8992-00171d21f472
- Toy: https://www.fab.com/listings/a356c0d1-0adc-4d59-b6c8-66ab157d1ffc